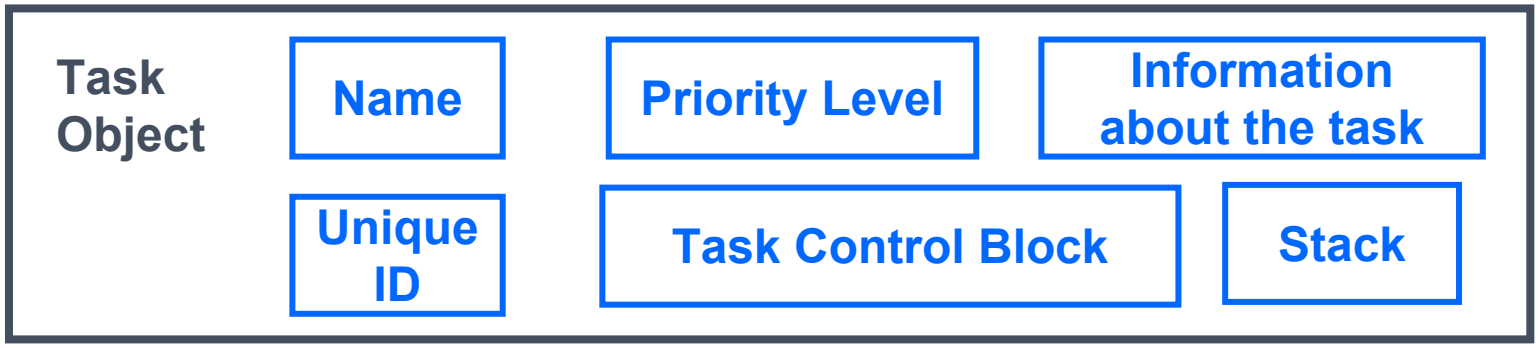
Chapter 4: Task Scheduling

Dr. Teja Mannepalli

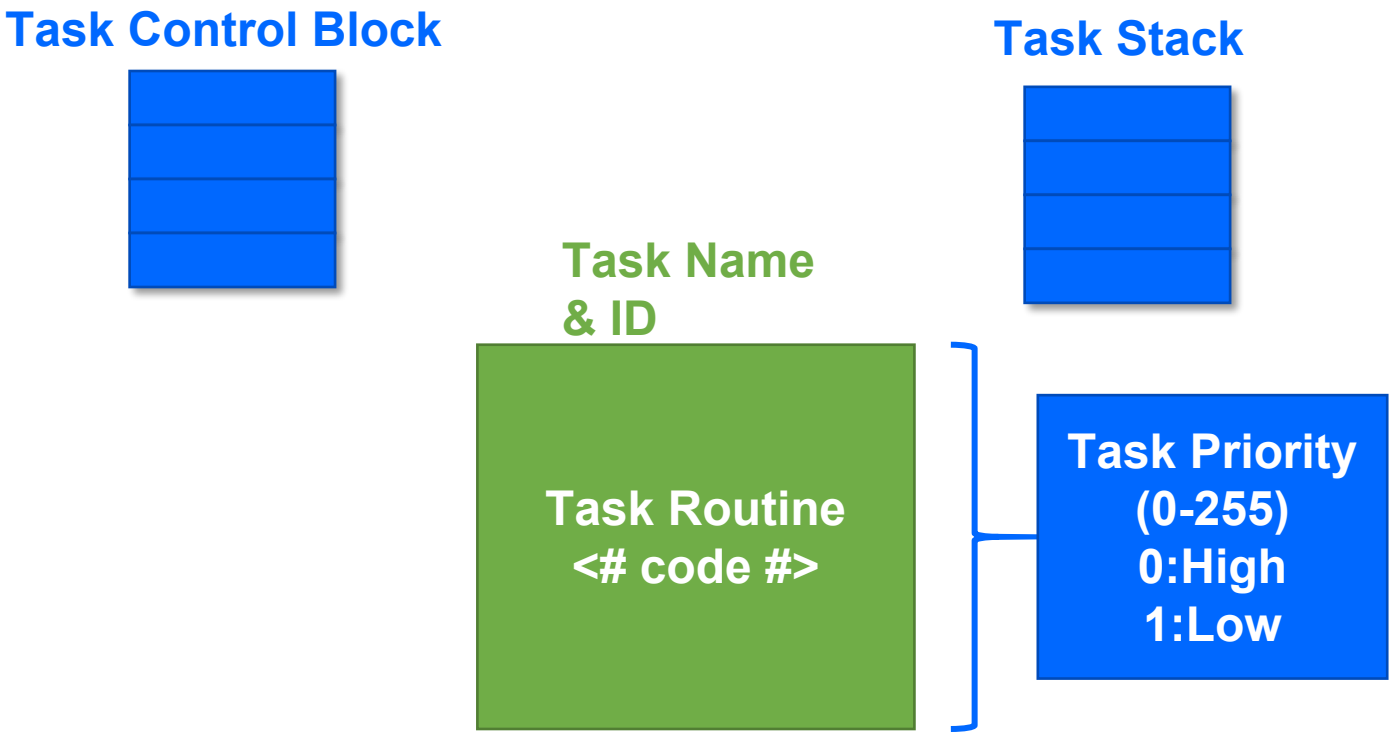
Assistant Professor,
Department of Electronics & Communications Engineering,
NIT Warangal

Task & Scheduling

- Task is a single unit of work (independent thread of execution) that needs to be completed within a specific timeframe, with strict deadlines. It competes for time.
- Two types of tasks in general:
  - Operating system tasks:
  - Application specific tasks:
  - System tasks (reserved priority):



Task Parameters and supporting structu



Task Control Block (TCB)

- A Task Control Block (TCB) is used for holding the information corresponding to a task. TCB usually contains the following set of information
  - *Task ID:* Task Identification Number
  - *Task State:* The current state of the task. (E.g. State= 'Ready' for a task which is ready to execute)
  - *Task Type*: Task type. Indicates what is the type for this task. The task can be a hard real time or soft real time or background task.
  - *Task Priority*: Task priority (E.g. Task priority =1 for task with priority = 1)
  - *Task Context Pointer*: Context pointer. Pointer for context saving.
  - *Task Memory Pointers*: Pointers to the code memory, data memory and stack memory for the task
  - *Task System Resource Pointers*: Pointers to system resources (semaphores, mutex etc) used by the task
  - *Task Pointers:* Pointers to other TCBs (TCBs for preceding, next and waiting tasks)
  - *Other Parameters* Other relevant task parameters

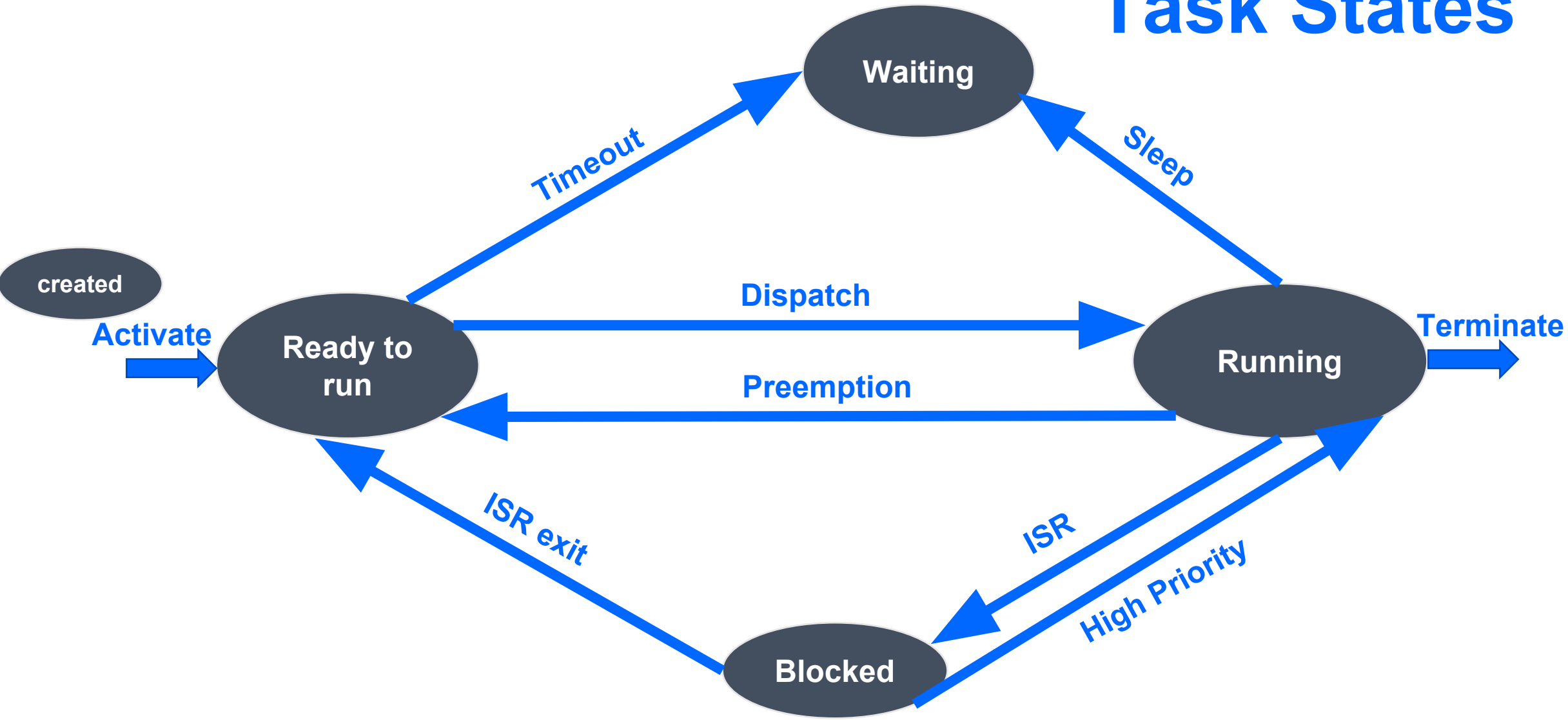
Kernel tasks

- The kernel has its own system tasks with priorities.
  - **Startup task:** when OS loads
  - **Exception handling task:** The handling of sudden events.
  - **Logging task:** log various system messages.
  - **Idle task:** will run when there is no other task to run.
  - **Debug agent task:** allows debugging with host debugger.

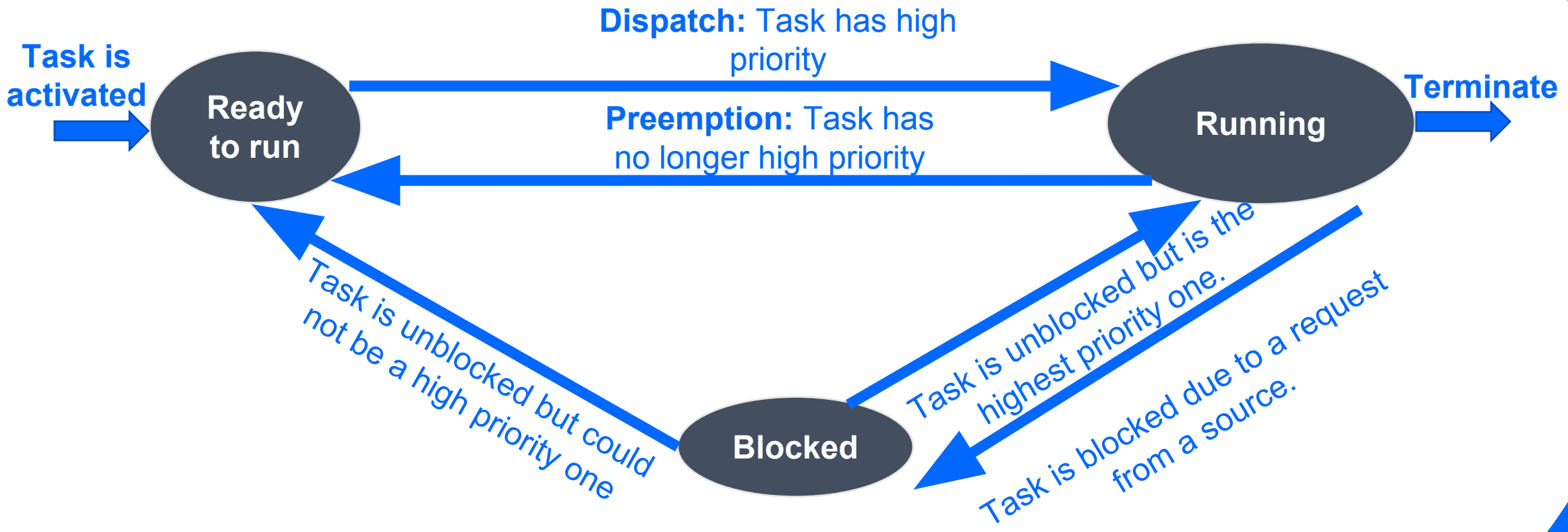
Task Scheduling

- The **critical tasks** do not allow to be interrupted by other tasks.
- The **task scheduling** is the mechanism to decide the task priorities.
- A **task state** is the current stage of a task in a process.
- In general (kernels can define their own states sometimes!) there are three main states:
  - **Ready state:** The task is ready but some high priority task is being executed currently.
  - **Blocked state:** The task could have requested a resource that is not available or has requested to wait until ISR complete or it delayed itself or etc.
  - **Running state:** the task is the highest priority and is running.

Task States


Waiting
Timeout
Sleep
created
Activate
Ready to run
Dispatch
Running
Terminate
Preemption
ISR exit
ISR
High Priority
Blocked

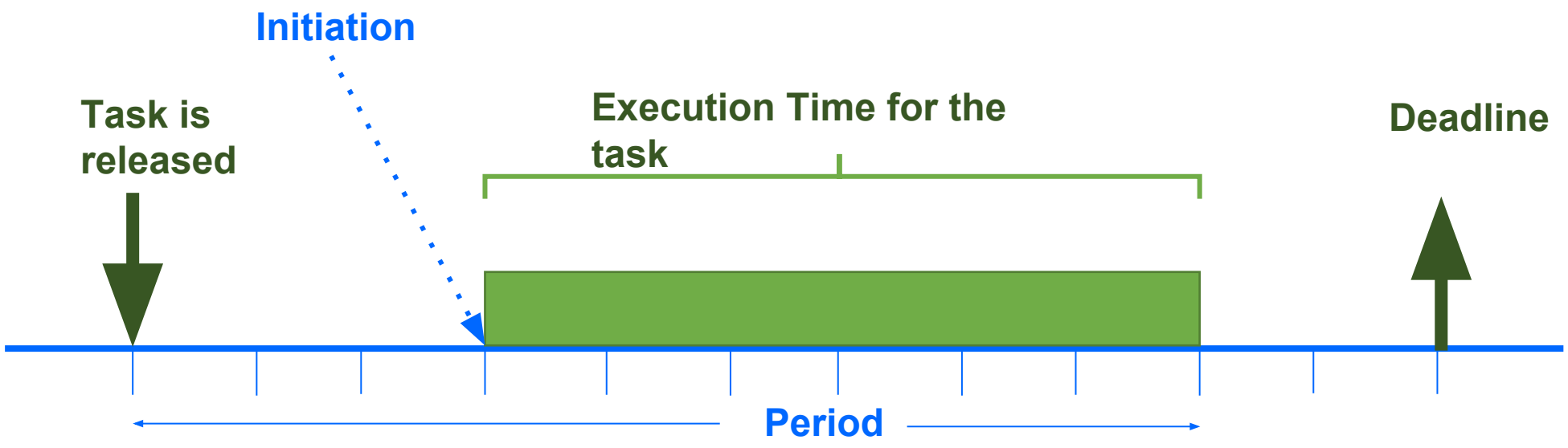
Another way of looking at it!



Task States

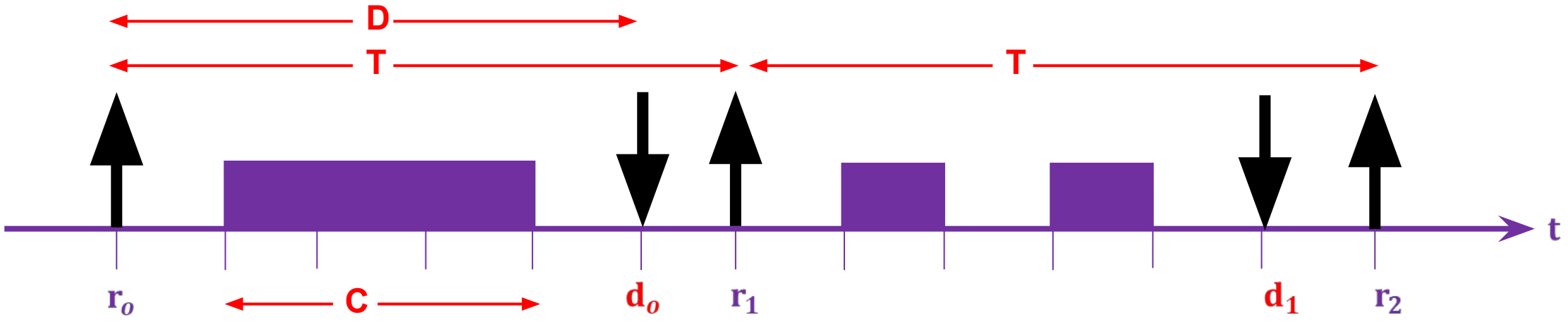
- **Created:** Into the memory.
- **Running:** The task is executed by the CPU.
- **Waiting**: The current task is in idle state for other task to be executed.
- **Ready to run:** The current task is waiting in queue for CPU time allocation.
- **Blocked:** The ISR should finish first. The current task in waiting mode.

Task Model



- Each task a triple: (execution time, period, deadline), Usually, deadline = period
- Can be initiated any time during the period

Task Model Parameters that you see every time!



Remember!

- r is task release time, i.e. the triggering time of the task execution request.
- C is task computation time when the processor is fully allocated to it.
- D is task relative deadline, i.e. the maximum acceptable delay for its processing.
- T is task period (valid only for periodic tasks).
- When the task has hard real-time constraints, the relative deadline allows computation of the absolute deadline

$$\mathrm{d} = \mathrm{r} + \mathrm{D}$$

- $\mathrm{r_k}$ is the release time of $(\mathrm{k}+1)^{th}$ request of task

$$\mathrm{r}_k = \mathrm{r}_o + kT$$

- d_k is the absolute deadline of the ${(k+1)}^{th}$ request of task

$$\mathrm{d}_k = \mathrm{r}_k + \mathrm{D}$$

- **Ready state:** The kernel puts a task in ready state when it is created and made ready to run.
- In this state, this task competes with all other ready tasks for CPU execution time.
- A **ready** state **cannot move** to the blocked state, it has to be moved to **running** state first, only then it **can be moved** to the **blocked** state.
- The kernel scheduler uses task priorities to determine the among the tasks to move to the running state.

Running State:

Unlike a ready task, a running task can move to the blocked state in any of the following ways:

- by making a call that requests an unavailable resource,
- by making a call that requests to wait for an event to occur, and
- by making a call to delay the task for some duration.
- In each of these cases, the task is moved from the running state to the blocked state.

TASK SCHEDULING ALGORITHMS

Task Scheduling

- Task scheduling is the action of assigning resources to perform tasks. It is the mechanism for deciding the task priority.
- The resources may be processors, network links or expansion cards. The tasks may be threads, processes or data flows.
- The scheduling activity is carried out by a mechanism called a scheduler.
- **Some kernels (most of) assign highest priority as 0 (lowest will be 255) or 255 (lowest will be 0).**

Pre-emptive and Non Pre-emptive Schedul

- In **preemptive scheduling**, the operating system can interrupt a running process to allocate the CPU to another process usually due to priority rules or time-sharing policies.
  - A process may be moved from Running → Ready state before it finishes.
- In **non-preemptive scheduling**, once a process starts using the CPU, it runs until it finishes or moves to a waiting state. The OS cannot forcibly take away the CPU.

- There are many task scheduling algorithms that are commonly used by the kernel to decide the order of execution.
- Depending on the requirement of the embedded system, the scheduling algorithm is chosen.
  - First In First Out
  - Round-robin algorithm
  - Round-robin with priority
  - Shortest job first
  - Non-preemptive multitasking
  - Preemptive multitasking

We will deal with!

- Round-robin with priority.
- First In First Out (FIFO)
- Preemptive Priority CPU Scheduling Algorithm.
- Rate Monotonic Analysis (RMA)
- Earliest Deadline First (EDF)

1. Round-Robin Preemptive Scheduling

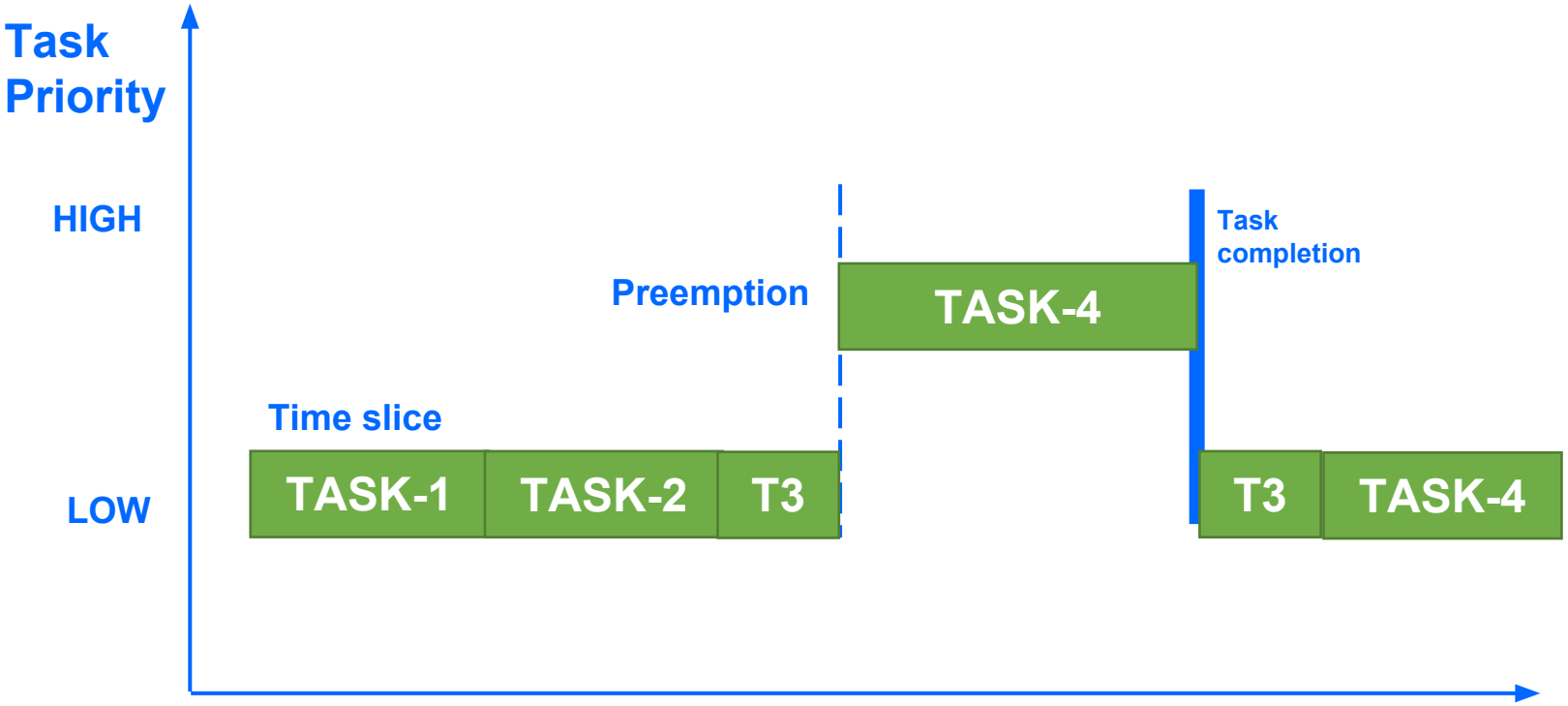
- RR algorithm uses time slices (also known as time quanta) are assigned to each process in equal portions and in circular order, handling all processes without priority.
  - The period of time for which a process is allowed to run in a preemptive multitasking system is generally called the *time slice* or *quantum*.
- Round-robin scheduling is simple, easy to implement, and starvation-free.
- Round-robin is a **pre-emptive** algorithm as the scheduler forces the process out of the CPU once the time quota expires.

Example (RR without priority)

- If we have three processes {A, B, C}, then the scheduler may run them in the sequence A, B, C, A, B, C, A, and so on, until they are all finished.

Process	Quantum 1	Quantum 2	Quantum 3	Quantum 4	Quantum 5
A	**Run**	Ready	Ready	**Run**	Ready
B	Ready	**Run**	Ready	Ready	**Run**
C	Ready	Ready	**Run**	Ready	Ready

Let us consider



Let us take a job with processes below

Process name	Arrival time (ms)	Execute time (ms)
P0	0	250
P1	50	170
P2	130	70
P3	190	100
P4	210	100

Quantum time is 100 ms

CPU execution time (in red)	RR scheduler	Notes
0	$P0_{250}$	P0 arrives & gets executed.
50	$P0_{200}$ $P1_{170}$	P1 arrives & waits
100	$P1_{170}$ $P0_{150}$	P0 is thrown from CPU as the execution time allotted is complete. P1 starts processing.
130	$P1_{140}$ $P0_{150}$ $P2_{70}$	P2 arrives.
190	$P1_{80}$ $P0_{150}$ $P2_{70}$ $P3_{100}$	P3 arrives.
200	$P0_{150}$ $P2_{70}$ $P3_{100}$ $P1_{70}$	P1 forced out of CPU and P0 gets processed.
210	$P0_{140}$ $P2_{70}$ $P3_{100}$ $P1_{70}$ $P4_{130}$	P4 arrives

CPU execution time (in red)	RR scheduler	Notes
300	$P2_{70}$ $P3_{100}$ $P1_{70}$ $P4_{130}$ $P0_{30}$	P0 thrown out & P2 gets executed.
370	$P3_{100}$ $P1_{70}$ $P4_{130}$ $P0_{30}$	P2 complete so P3 is started.
400	$P1_{70}$ $P3_{70}$ $P4_{130}$ $P0_{30}$	P3 is thrown aside and P1 is started.
470	$P3_{70}$ $P4_{130}$ $P0_{30}$	P1 completes. P3 is continued.
500	$P4_{100}$ $P3_{70}$ $P0_{30}$	P3 is halted. P4 starts.
600	$P3_{70}$ $P0_{30}$	P4 completes & P3 continued again.
700	Complete	complete

2. First Come First Serve (FCFS)

- First Come, First Serve (FCFS) as the name suggests, processes are attended to in the order in which they arrive in the ready queue,
  - like you lining up at my room for your answer scripts!
- FCFS Scheduling is a **non-preemptive** algorithm, meaning once a process starts running, it cannot be stopped until it voluntarily relinquishes the CPU, typically when it terminates or performs I/O.
- The average waiting time is not minimal.
- This method schedules processes in the order they arrive, without considering priority or other factors.

Terminology

- Completion Time (CT): This is the time when the process completes its execution.
- Arrival Time (AT): This is the time when the process has arrived in the ready state.
- Service Time: time at which a process starts execution on the CPU.
- Burst Time (BT): total time required by a process for execution on the CPU.
- Turnaround Time (TAT) means the total time a process takes to complete.
- Waiting Time (WT) represents the time a process spends waiting in the ready queue before it gets executed on the CPU
  - $Turnaround\ Time\ (TAT) = Completion\ Time - Arrival\ Time$
  - $Waiting\ Time\ (WT) = Turnaround\ Time - Burst\ Time$

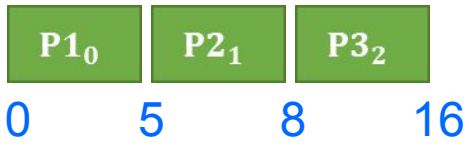
Example!

Process	Burst Time (BT)	Arrival Time (AT)
P1	5 ms	0 ms
P2	3 ms	1 ms
P3	8 ms	2 ms

The table of arrival time and CPU time for three processes P1, P2 and P3

1. Completion Time (CT)
2. Turnaround Time (TAT = CT – AT)
3. Waiting Time (WT = TAT – BT)

$P\ \#id_{AT}$



Process	Burst Time (BT)	Arrival Time (AT)	Completion Time (CT)	TAT	WT	Service Time (ST)
P1	5 ms	0 ms	5 ms	5 ms	0 ms	0 ms
P2	3 ms	1 ms	8 ms	7 ms	4 ms	5 ms
P3	8 ms	2 ms	16 ms	14 ms	6 ms	8 ms

Try this!

Process	Burst Time (BT)	Arrival Time (AT)
P1	5 ms	0 ms
P2	4 ms	1 ms
P3	10 ms	2 ms
P4	6 ms	3 ms

3. Preemptive Priority CPU Scheduling Algorithm

- It is based on the priority of a process.
  - Low priority → last of execution
  - High priority → first of execution
- Step-1: Check the process whose arrival time is zero. It will be executed until a new process comes up.
- Step-2: Check the priority (P_i) of the next available process (i).
  - If $\mathrm{P}_i > \mathrm{P}_{i-1}$ → Execute i^{th} process.
  - If $\mathrm{P}_i < \mathrm{P}_{i-1}$ → Execute $i - 1^{th}$ process.
  - If $\mathrm{P}_i = \mathrm{P}_{i-1}$ → Execute process with least arrival time.
- Choose the process which is having the highest priority & execute it.

Example!

Process	Arrival Time	Burst Time	Priority
P1	0 ms	3 ms	5
P2	1 ms	2 ms	8 (low)
P3	2 ms	1 ms	2

At t = 0 ms,

1. P1 arrives, and get executed.

At t = 1 ms,

2. P2 arrives but has low priority. So, P1 will get executed again.

At t = 2 ms,

2. P3 arrives but has high priority. So, P3 will get executed and P2 is halted.

At t = 0 ms,

- P1 arrives, and get executed.

CPU time instance	Process	Execution Time	Burst Time left
t = 0 ms to 1 ms	P1	1 ms	2 ms

At t = 1 ms,

2. P2 arrives but has low priority. So, P1 will get executed again.

CPU time instance	Process	Execution Time	Burst Time left
t = 1 ms to 2 ms	P1	2 ms	1 ms

At t = 2 ms,

2. P3 arrives but has high priority. So, P3 will get executed and P2 is halted.

CPU time instance	Process	Execution Time	Burst Time left
t = 2 ms to 3 ms	P3	1 ms	0 ms

Example!

Process	Arrival Time	Burst Time	Priority
P1	0 ms	3 ms	5
P2	1 ms	2 ms	2
P3	2 ms	1 ms	8

Processor utilization factor (U)

- The U of a set of n periodic tasks is:

$$\mathrm{U} = \sum_{i=1}^{n} \mathrm{U}_i = \sum_{i=1}^{n} \frac{\mathrm{C}_i}{\mathrm{T}_i} = n(2^n - 1)$$

- A major cycle of a set of tasks is an interval of time on the time line such that in each major cycle, the different tasks recur identically.

$$Major\ Cycle = [0, LCM(\mathrm{T}_i)], i = 1 \ldots n$$

4. Rate Monotonic Analysis (RMA)

- RMA is a task scheduling algorithm that is:
  - Preemptive (HP task can interrupt in between),
  - static (the priority assigned to tasks before execution is fixed, and do not change over time.)
  - and priority-based algorithm.
- For a set of periodic tasks, assigning the priorities according to the rate monotonic (RM) algorithm means that *tasks with shorter periods (higher request rates) get higher priorities.*

$$Shorter\ periods \rightarrow higher\ priority.$$

Sufficient Schedulability Condition of RM

- A task is schedulable if all its deadlines are met (i.e., the task completes its execution before its period elapses.)
- For a set of n periodic tasks with unique periods ($D = T$), a feasible schedule that will always meet deadlines exists if the CPU utilization is below a specific bound. The schedulability test for RMS is:

$$\mathrm{U} = \sum_{i=1}^{n} \mathrm{U}_i = \sum_{i=1}^{n} \frac{\mathrm{C}_i}{\mathrm{T}_i} = n(2^n - 1)$$

5. Earliest Deadline First (EDF)

- The EDF is a task scheduling algorithm that is:
  - dynamic (priorities are assigned to tasks based on dynamic parameters that may change during task execution. Priorities are variable in run-time)
  - priority based
  - Pre-emptive based
- The earliest deadline first (EDF) algorithm assigns priority to tasks according to their absolute deadline: the task with the earliest deadline will be executed at the highest priority.

Sufficient Schedulability Condition of EDF

- A task is schedulable if all its deadlines are met (i.e., the task completes its execution before its period elapses.)
- For a set of n periodic tasks with unique periods ($D = T$), to be schedulable with the EDF algorithm if and only if the processor utilization factor is less than or equal to 1:

$$\mathrm{U} = \sum_{i=1}^{n} \mathrm{U}_i \leq 1$$

THANK YOU

For your patience!